

클로드 코드, 코덱스 통합 방법

▼ 클로드 코드와 코덱스 통합 구조 추천 (5.5 TX)

결론: 네 문제의식이 맞습니다

네가 말한 우려는 정확해.

처음 내가 제안한 구조는 **여행계획 테스트용 v2** 구조로는 괜찮습니다.

```
agents/          ← 공통 역할 원본
.claude/agents/  ← Claude Code용 sub-agent wrapper
```

하지만 작업이 커지면 문제가 생길 수 있습니다.

- `agents/researcher.md` 에도 지침이 있고
- `.claude/agents/trip-researcher.md` 에도 지침이 있고
- 나중에 `.codex/...` 에도 비슷한 지침이 생기면
- 어느 쪽이 진짜 원본인지 헷갈릴 수 있음
- 수정할 때마다 세 군데를 봐야 함
- 한쪽만 수정하면 drift, 즉 지침 불일치가 생김

그래서 장기적으로는 단순한 **“공통 agents + Claude wrapper”**보다, 아래처럼 생각하는 게 더 좋습니다.

공통으로 가져갈 것은 **“역할 문서”**가 아니라 **“계약서”**여야 한다.

Claude/Codex별 에이전트 파일은 그 계약서를 실행하는 **“어댑터”**가 되어야 한다.

1. 네 제안: `.claude`, `.codex` 로 모델별 에이전트를 분리하자

판단

나는 이 방향에 **대체로 찬성**합니다.

다만 조건이 있습니다.

항목	판단
Claude 전용 sub-agent는 <code>.claude/agents/</code> 에 둔다	찬성
Codex 전용 설정은 <code>.codex/</code> 또는 Codex가 인식하는 위치에 둔다	찬성
orchestrator는 공통으로 둔다	찬성
skills도 일부 공통으로 둔다	대체로 찬성
모든 agent 지침을 Claude/Codex별로 완전히 따로 쓴다	비추천
공통 원본 없이 도구별 지침만 관리한다	장기적으로 위험

즉, 네 방향은 좋지만, **완전 분리**가 아니라 **계층 분리**가 더 안전합니다.

2. 왜 “완전 분리”는 위험한가

네가 말한 구조를 극단적으로 가면 이렇게 됩니다.

```
trip-planning-harness/
├── orchestrator/
├── skills/
├── .claude/
│   └── agents/
│       ├── researcher.md
│       └── judge.md
└── .codex/
```

```

├── agents/
│   ├── researcher.md
│   └── judge.md
└── artifacts/

```

이 방식은 겉으로는 깔끔해 보여.

그런데 시간이 지나면 이런 문제가 생깁니다.

문제	설명
의미가 갈라짐	Claude researcher와 Codex researcher가 조금씩 다른 일을 하게 됨
평가가 어려움	Claude 결과와 Codex 결과가 다른 이유가 "모델 차이"인지 "지침 차이"인지 알기 어려움
수정 비용 증가	같은 업무 규칙을 Claude용, Codex용으로 각각 수정해야 함
테스트 불가능성 증가	어느 지침이 최신인지 확인하기 어려움
하네스 품질이 도구별로 분화	결국 Claude 하네스와 Codex 하네스가 서로 다른 제품이 됨

특히 네 목적은 "공통 지침을 유지하되, 실행 시 Claude Code 또는 Codex를 선택할 수 있는 구조"야.

그렇다면 완전 분리된 목적과 약간 어긋납니다.

3. 더 좋은 기준: "공통 원본"을 agent가 아니라 contract로 둔다

내가 지금 제안하고 싶은 더 나은 구조는 이겁니다.

```

trip-planning-harness/
├── AGENTS.md
├── CLAUDE.md
├── harness/
│   ├── ORCHESTRATOR.md
│   ├── contracts/
│   │   ├── intake.contract.md
│   │   ├── research.contract.md
│   │   ├── planning.contract.md
│   │   └── judge.contract.md
│   ├── skills/
│   │   ├── intake.md
│   │   ├── research.md
│   │   ├── planning.md
│   │   └── judge.md
│   ├── rubrics/
│   │   └── judge-rubric.md
│   └── schemas/
│       ├── intake.schema.md
│       ├── research.schema.md
│       └── final-plan.schema.md
├── .claude/
│   ├── settings.local.json
│   └── agents/
│       ├── trip-intake.md
│       ├── trip-researcher.md
│       ├── trip-planner.md
│       └── trip-judge.md
├── .codex/
│   ├── config.toml
│   └── agents/
│       ├── skills/
│       │   ├── trip-research/
│       │   │   └── SKILL.md
│       │   ├── trip-judge/
│       │   │   └── SKILL.md
└── artifacts/

```

핵심은 `contracts/` 입니다.

4. contract가 뭐냐

비개발자 관점에서 contract는 "**업무 위임서 + 납품 기준서**"라고 보면 됩니다.

예를 들어 `research.contract.md` 에는 이런 것이 들어갑니다.

Research 단계의 목적:

- 여행 후보지, 이동, 운영시간, 비용, 가족 적합성 정보를 조사한다.

입력:

- artifacts/{run_id}/00_intake.md

출력:

- artifacts/{run_id}/01_research_brief.md

반드시 포함:

- 후보지명
- 위치
- 이동 시간
- 운영시간
- 예상 비용
- 가족/아이 동반 적합성
- 불확실한 정보
- 출처

금지:

- 최종 일정을 작성하지 않는다.
- 검증되지 않은 정보를 확정처럼 쓰지 않는다.
- 사용자의 핵심 제약을 무시하지 않는다.

완료 기준:

- 후보지 최소 N개
- 각 후보지마다 이동/시간/비용/리스크 포함
- 불확실성 분리

이건 Claude용도 아니고 Codex용도 아닙니다.

그냥 이 업무가 무엇인지 정의한 원장입니다.

5. 그러면 `.claude/agents/` 는 무엇을 하느냐

`.claude/agents/trip-researcher.md` 는 이제 “업무 내용 원본”이 아닙니다.

Claude Code에서 sub-agent로 등록되기 위한 **실행 어댑터**입니다.

```
---
name: trip-researcher
description: Use during the research phase of the trip-planning harness. Researches candidates, transportation, opening hours, costs, and family suitability before itinerary drafting.
tools: Read, Grep, Glob, WebSearch, WebFetch
model: sonnet
---
```

You are the Claude Code sub-agent for the trip research phase.

Read and follow:

- harness/contracts/research.contract.md
- harness/skills/research.md
- harness/ORCHESTRATOR.md

Input:

- artifacts/{run_id}/00_intake.md

Output:

- artifacts/{run_id}/01_research_brief.md

Do not write the final itinerary.
Return only a concise handoff summary to the parent agent.

여기에는 Claude Code 특화 요소만 들어갑니다.

Claude 전용 요소	예
<code>name</code>	Claude Code에서 호출될 sub-agent 이름
<code>description</code>	Claude가 언제 이 agent를 호출할지 판단하는 설명
<code>tools</code>	Read, WebSearch, WebFetch 등
<code>model</code>	sonnet, opus, haiku 등
parent agent에 반환하는 방식	“요약만 반환”

Claude Code 공식 문서도 project subagent는 `.claude/agents/`에 둘 수 있고, subagent는 Markdown 파일과 YAML frontmatter로 정의되며, 별도 prompt와 tool permission을 갖는다고 설명합니다. 또한 Claude는 subagent의 description을 보고 언제 위임할지 판단합니다. (Claude Code)

6. 그러면 Codex 쪽은 어떻게 하느냐

Codex는 Claude Code와 구조가 다릅니다.

Codex에서는 `AGENTS.md`가 durable project guidance 역할을 하고, 전역/프로젝트/하위 폴더별 `AGENTS.md`를 계층적으로 읽습니다. Codex 공식 문서는 파일이 루트에서 현재 작업 디렉터리까지 합쳐지고, 가까운 디렉터리의 지침이 뒤에 붙어 더 강하게 작동한다고 설명합니다. (OpenAI Developers)

Codex도 subagent workflow를 지원하지만, 문서상 중요한 차이가 있습니다.

- Codex는 현재 subagent를 명시적으로 요청했을 때만 새 agent를 생성합니다.
- 여러 agent가 돌면 Codex가 결과를 기다린 뒤 통합 응답을 반환합니다.
- subagent는 별도의 모델·도구 작업을 하기 때문에 단일 agent보다 토큰을 더 씹니다. (OpenAI Developers)

그리고 Codex의 재사용 workflow 쪽은 **Skills**가 강합니다. Codex Skills는 `.agents/skills`에 둘 수 있고, `SKILL.md`는 `name`과 `description`을 포함해야 하며, Codex는 처음에는 skill metadata만 보고 필요할 때 전체 `SKILL.md`를 읽는 progressive disclosure 구조를 씹니다. (OpenAI Developers)

그래서 Codex 쪽은 보통 이렇게 설계하는 게 자연스럽습니다.

```
AGENTS.md          ← Codex가 항상 읽는 얇은 공통 진입점
.agents/skills/trip-research/
└── SKILL.md        ← Codex용 reusable workflow
.codex/config.toml  ← Codex 실행 설정, 모델, sandbox, profile 등
```

7. 중요한 결론: Claude와 Codex는 “같은 구조”로 강제하면 안 된다

여기서 많은 사람이 실수할 수 있습니다.

잘못된 접근	왜 문제인가
Claude의 <code>.claude/agents/</code> 와 똑같이 Codex에도 <code>.codex/agents/</code> 를 만들기	Codex가 그 구조를 Claude와 똑같이 인식한다는 보장이 없음
두 도구의 agent 파일을 1:1로 맞추려 함	각 도구가 호출·등록·권한·skill을 다르게 처리함
모든 공통 지침을 하나의 큰 Markdown에 몰아넣음	컨텍스트가 비대해지고, 필요한 지침만 읽는 장점이 사라짐
Claude/Codex별로 완전히 다른 지침을 둬	모델 비교가 어려워지고 drift 발생

더 좋은 접근은 이것입니다.

```
공통 업무 계약서:
- 무엇을 해야 하는가
- 입력은 무엇인가
- 출력은 무엇인가
- 성공 기준은 무엇인가
- 금지사항은 무엇인가

Claude 어댑터:
- Claude Code에서 이 업무를 sub-agent로 어떻게 호출할 것인가

Codex 어댑터:
- Codex에서 이 업무를 AGENTS.md / Skill / 명시 subagent workflow로 어떻게 실행할 것인가
```

8. 개발 업계에서 흔히 쓰는 방식과 연결하면

이 문제는 사실 AI 하네스만의 문제가 아닙니다.

개발 업계에서는 오래전부터 비슷한 문제를 이렇게 처리해왔습니다.

개발 업계 개념	하네스에 대응
Single Source of Truth	공통 contract / schema / rubric
Adapter Pattern	<code>.claude/agents/</code> , <code>.agents/skills</code> , <code>.codex/config.toml</code>
Interface / Contract	각 phase의 입력·출력·완료 기준
DRY 원칙	같은 규칙을 여러 파일에 복붙하지 않기
Configuration layering	전역 설정, 프로젝트 설정, 로컬 설정 분리
Generated files	필요하면 공통 spec에서 도구별 파일 자동 생성
Tests / Evals	결과물이 contract를 지켰는지 검사

Codex 공식 Best Practices에서도 `AGENTS.md` 는 짧고 정확하게 유지하고, 너무 커지면 planning, code review, architecture 같은 task-specific Markdown 파일을 참조하라고 권합니다. ([OpenAI Developers](#))

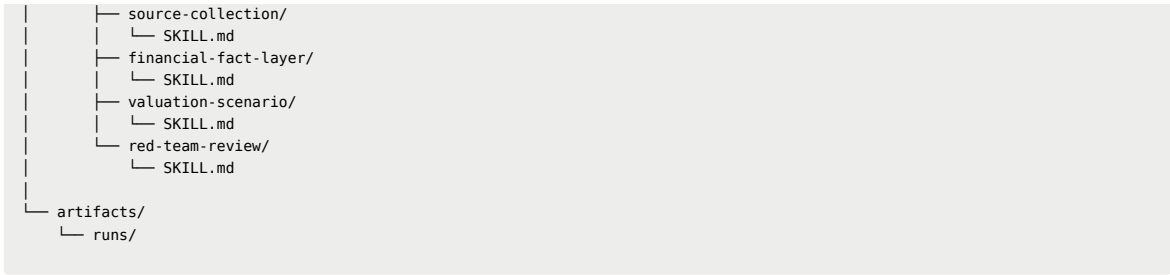
Claude Code 설정 문서도 user/project/local scope를 구분하고, subagents는 사용자 범위와 프로젝트 범위 위치가 다르며, settings는 scope별 precedence를 갖는다고 설명합니다. ([Claude Code](#))

또 AGENTS.md 오픈 포맷 설명은 `AGENTS.md` 를 “agent를 위한 README”로 설명하고, 많은 도구에서 쓰이는 예측 가능한 지침 위치로 제시합니다. 대형 프로젝트에서는 하위 폴더별 `AGENTS.md` 를 두어 가까운 지침이 우선하도록 하는 패턴도 설명합니다. ([Agents](#))

9. 내가 추천하는 최종 구조

네 목적에 가장 맞는 구조는 이쪽입니다.

```
investment-research-harness/
├── AGENTS.md
├── CLAUDE.md
├── harness/
│   ├── README.md
│   ├── ORCHESTRATOR.md
│   ├── contracts/
│   │   ├── 00_intake.contract.md
│   │   ├── 01_source_collection.contract.md
│   │   ├── 02_fact_layer.contract.md
│   │   ├── 03_financial_model.contract.md
│   │   ├── 04_moat_analysis.contract.md
│   │   ├── 05_risk_review.contract.md
│   │   ├── 06_red_team.contract.md
│   │   └── 07_final_memo.contract.md
│   ├── skills/
│   │   ├── source_collection.md
│   │   ├── financial_fact_layer.md
│   │   ├── valuation_scenario.md
│   │   ├── moat_analysis.md
│   │   └── red_team_review.md
│   ├── rubrics/
│   │   ├── source_quality.md
│   │   ├── valuation_quality.md
│   │   └── final_memo_rubric.md
│   └── schemas/
│       ├── fact_layer.schema.md
│       ├── valuation_table.schema.md
│       └── final_memo.schema.md
├── .claude/
│   ├── settings.json
│   ├── settings.local.json
│   └── agents/
│       ├── source-collector.md
│       ├── financial-fact-agent.md
│       ├── valuation-agent.md
│       ├── moat-agent.md
│       ├── red-team-agent.md
│       └── final-review-agent.md
├── .codex/
│   └── config.toml
├── .agents/
│   └── skills/
```



10. 왜 이 구조가 더 낫나

영역	역할	공통/전용
<code>harness/contracts/</code>	각 단계의 목적, 입력, 출력, 완료 기준	공통
<code>harness/skills/</code>	업무 절차, 분석 방법론, 금지사항	공통
<code>harness/rubrics/</code>	평가 기준	공통
<code>harness/schemas/</code>	산출물 형식	공통
<code>.claude/agents/</code>	Claude Code sub-agent 등록·도구·모델·호출 설명	Claude 전용
<code>.agents/skills/</code>	Codex가 발견할 reusable workflow	Codex 전용
<code>.codex/config.toml</code>	Codex 모델·sandbox·profile 설정	Codex 전용
<code>AGENTS.md</code>	Codex 및 다중 에이전트용 기본 안내	공통에 가까움
<code>CLAUDE.md</code>	Claude Code 진입점	Claude 전용
<code>artifacts/</code>	결과물 저장	공통

이 구조에서는 중요한 규칙이 이겁니다.

- 업무 의미는 `harness/` 에 둔다.
- 도구별 실행 방식은 `.claude/` , `.codex/` , `.agents/` 에 둔다.

11. 그러면 `agents/` 공통 폴더는 없애도 되나?

장기 구조에서는 나는 없애거나 이름을 바꾸는 쪽이 낫다고 봅니다.

기존:

```

agents/
  researcher.md
  judge.md
  
```

이 이름은 혼란을 만듭니다.

왜냐하면 Claude Code에는 진짜 sub-agent가 `.claude/agents/` 에 있기 때문입니다.

그래서 공통 폴더 이름을 바꾸는 게 좋습니다.

기존 이름	추천 이름	이유
<code>agents/</code>	<code>contracts/</code>	역할보다 입출력 계약을 강조
<code>agents/</code>	<code>roles/</code>	“실행 agent”가 아니라 “역할 정의”임을 명확히 함
<code>agents/</code>	<code>role_specs/</code>	도구별 agent와 구분됨
<code>skills/</code>	<code>methods/</code> 또는 <code>procedures/</code>	Claude/Codex native skill과 혼동 방지

내 추천은:

```

harness/contracts/
harness/procedures/
harness/rubrics/
harness/schemas/
  
```

이렇게 바꾸는 것입니다.

특히 `skills/` 도 조심해야 합니다.

Claude와 Codex 모두 "Skill"이라는 기능을 갖고 있기 때문에, 공통 폴더를 그냥 `skills/` 라고 부르면 나중에 헷갈릴 수 있습니다.

12. skills도 모델별로 따로 뒀어 하나?

내 판단

종류	공통으로 둘지	이유
분석 방법론	공통	모델과 무관한 업무 지식
출력 형식	공통	Claude/Codex 결과 비교를 위해 같아야 함
금지사항	공통	신뢰성 기준은 모델별로 달라지면 안 됨
도구 사용법	모델별	Claude와 Codex의 tool/permission 방식이 다름
호출 조건	모델별	Claude는 subagent description, Codex는 skill description/명시 호출 방식이 다름
모델 선택	모델별	Claude는 sonnet/opus/haiku, Codex는 GPT 계열
sandbox/permission	모델별	실행 환경 설정이 다름

즉:

업무 방법론은 공통.

도구 실행법은 모델별.

그래서 공통 `skills/` 라는 이름보다 `procedures/` 가 더 안전합니다.

```
harness/procedures/valuation_scenario.md    ← 공통 분석 절차
.claude/skills/...                          ← Claude native skill
.agents/skills/...                          ← Codex native skill
```

13. 복잡한 투자 리서치 하네스라면 이렇게 나누는 게 좋다

공통 계약

```
harness/contracts/02_fact_layer.contract.md
```

여기에 들어갈 내용:

- 입력 자료 범위
- 반드시 수집할 재무 항목
- 음수 표기 방식
- 단위 표기 방식
- 출처 필수 여부
- 불확실성 표시 방식
- 출력 파일 경로
- 완료 기준

Claude 전용 agent

```
.claude/agents/financial-fact-agent.md
```

여기에 들어갈 내용:

- Claude Code sub-agent name
- description
- tools
- model
- 읽어야 할 공통 contract
- 쓰기 권한
- parent에게 반환할 요약 방식

Codex 전용 skill

```
.agents/skills/financial-fact-layer/SKILL.md
```

여기에 들어갈 내용:

- Codex skill name
- description
- 언제 활성화할지
- 공통 contract를 읽으라는 지시
- Codex 환경에서 사용할 명령/스크립트
- 결과 저장 규칙

14. 가장 중요한 운영 원칙

원칙 1. 공통 원본은 “업무 의미”만 가진다

공통 파일에는 이런 것이 들어갑니다.

```
무엇을 해야 하는가
무엇을 읽어야 하는가
무엇을 만들 것인가
좋은 결과의 기준은 무엇인가
어떤 실수를 하면 안 되는가
```

공통 파일에는 이런 것을 넣지 않는 게 좋습니다.

```
Claude의 tools 필드
Codex의 config.toml 설정
Claude sub-agent model
Codex sandbox 설정
특정 플랫폼 호출 문법
```

원칙 2. 도구별 파일은 “실행 방식”만 가진다

Claude 파일에는 Claude 실행 방식만:

```
name
description
tools
model
permissionMode
읽을 contract 경로
출력 경로
```

Codex 파일에는 Codex 실행 방식만:


```
AGENTS.md
config.toml
.agents/skills/*/SKILL.md
명시적 subagent 실행 지시
sandbox/profile 설정
```

원칙 3. 같은 문장을 두 곳에 길게 복붙하지 않는다

가장 위험한 구조는 이것입니다.

```
harness/procedures/research.md
.claude/agents/researcher.md
.agents/skills/research/SKILL.md
```

세 파일에 모두 “조사 절차 20줄”이 들어가는 것.

이러면 반드시 불일치가 생깁니다.

대신 이렇게 해야 합니다.

```
harness/procedures/research.md
```

여기에 조사 절차 원본을 두고,

```
.claude/agents/researcher.md
.agents/skills/research/SKILL.md
```

에는 “위 파일을 읽고 따르라”고만 둡니다.

15. 최종 추천안

여행계획 테스트 v2

지금처럼 해도 됩니다.

```
agents/
skills/
prompts/
.claude/agents/
```

테스트용으로 충분합니다.

투자 리서치/기업 비교 같은 본격 v3

이 구조로 바꾸는 걸 추천합니다.

```
harness/
  contracts/
  procedures/
  rubrics/
  schemas/

.claude/
  agents/
  skills/      ← 필요할 때만

.codex/
  config.toml

.agents/
  skills/      ← Codex용 reusable workflow
```

16. 최종 판단

네 생각 중 맞는 부분:

네 생각	판단
복잡해질수록 공통 agents 원본은 위험할 수 있다	맞음
Claude/Codex별 실행 구조를 분리해야 한다	맞음
<code>.claude</code> , <code>.codex</code> 같은 전용 계층이 필요할 수 있다	맞음
orchestrator와 skills는 공통으로 둘 수 있다	맞음

조정하면 더 좋은 부분:

네 생각	보완
서브 에이전트는 모델별로 완전히 따로 둔다	“실행 어댑터”는 따로 두되, 업무 계약은 공통으로 둔다
skills도 모델별로 따로 돌지 고민	분석 절차는 공통, native skill packaging은 모델별
공통 원본이 안 좋을 수 있다	공통 “역할 원본”은 애매하지만, 공통 “contract/schema/rubric”은 매우 중요
복잡해지면 분리한다	분리하되, 의미와 실행을 분리해야 함

한 줄 결론

장기적으로는 `agents/` 공통 원본을 중심에 두기보다, `harness/contracts + procedures + rubrics + schemas`를 공통 원장으로 두고, `.claude/agents`와 `.agents/skills`는 각 도구가 그 원장을 실행하기 위한 얇은 adapter로 두는 구조가 가장 안전합니다.

▼ 통합구조로 하네스 구축 시 사용할 LLM 선택 (5.5 TX)

결론: “둘에게 각각 전체 하네스를 만들게 하는 방식”은 비추천입니다

네 걱정이 맞습니다.

Claude Code에게 전체를 맡기면:

- `.claude/agents/`는 잘 만들 가능성이 높음
- 하지만 Codex용 `AGENTS.md`, `.agents/skills/`, `.codex/config.toml`의 실제 작동 방식까지 정확히 맞추지는 불확실함

반대로 Codex에게 전체를 맡기면:

- Codex용 `AGENTS.md`, Skills 구조는 잘 만들 가능성이 높음
- 하지만 Claude Code의 `.claude/agents/` sub-agent 구조, tools, model, description, session loading 규칙을 정확히 반영하지 못할 수 있음

그래서 가장 안전한 방식은 이것입니다.

공통 하네스 본체는 하나만 만들고, Claude Code와 Codex는 각각 자기 adapter만 만들거나 검토하게 한다.

즉, 전체 작업을 두 번 하는 게 아니라:

공통 본체 1개
+ Claude Code용 실행 껍데기
+ Codex용 실행 껍데기

로 가는 게 좋습니다.

1. 하지 않는 게 좋은 방식

방식 A: Claude Code에게 전체를 다 만들게 하기

```
Claude Code:
- 공통 harness 만들기
- .claude/agents 만들기
- .codex/config 만들기
- .agents/skills 만들기
```

이 방식은 빠르지만 위험합니다.

위험	이유
Claude 친화 구조로 기술 가능성	Claude Code가 자기 생태계 기준으로 설계할 수 있음
Codex adapter가 컴데기만 그럴듯할 수 있음	실제 Codex에서 trigger, skill loading, AGENTS.md 계층이 맞는지 검증 필요
나중에 Codex로 실행할 때 오류 발견	구조는 있어도 실제 호출 방식이 어긋날 수 있음

Claude Code sub-agent는 `.claude/agents/` 에 Markdown + YAML frontmatter 형식으로 정의되고, project subagent 위치도 `.claude/agents/` 로 정해져 있습니다. 또한 `/agents` 인터페이스가 권장 관리 방식이고, subagent 파일은 세션 시작 시 로드됩니다. 이건 Claude Code 고유 규칙입니다. (Claude Code)

방식 B: Codex에게 전체를 다 만들게 하기

```
Codex:
- 공통 harness 만들기
- .codex/config 만들기
- .agents/skills 만들기
- .claude/agents 만들기
```

이것도 반대 방향으로 위험합니다.

위험	이유
Codex 친화 구조로 기술 가능성	Codex는 AGENTS.md와 Skills 중심으로 생각할 수 있음
Claude Code sub-agent가 실제 등록되지 않을 수 있음	YAML frontmatter, description, tools, model 규칙이 어긋날 수 있음
Claude Code에서 자동 위임이 안 될 수 있음	description이 애매하면 sub-agent가 호출되지 않음

Codex Skills는 `.agents/skills/.../SKILL.md` 처럼 skill directory와 `SKILL.md` 를 기반으로 하며, `name` 과 `description` 이 필요합니다. Codex는 처음부터 전체 skill 본문을 다 읽는 것이 아니라, skill의 이름·description·경로를 먼저 보고 필요한 경우 전체 `SKILL.md` 를 읽는 progressive disclosure 방식을 사용합니다. (OpenAI Developers)

방식 C: Claude용 하네스와 Codex용 하네스를 아예 따로 만들기

이건 단기적으로는 안전해 보이지만, 장기적으로는 비추천입니다.

문제	설명
사실상 두 제품이 됨	Claude 하네스와 Codex 하네스가 점점 다르게 진화
결과 비교가 어려움	결과 차이가 모델 차이인지 지침 차이인지 알기 어려움
유지보수 비용 증가	같은 규칙을 두 번 고쳐야 함
오류 가능성 증가	한쪽만 최신화되고 다른 쪽은 낡을 수 있음

네 목적은 “같은 하네스를 Claude Code 또는 Codex로 선택 실행”하는 것이므로, 완전 분리는 마지막 선택지로 보는 게 좋습니다.

2. 추천 방식: “공통 본체 먼저, adapter는 각 도구가 검토”

가장 안전한 구축 순서는 아래입니다.

```
1단계: 공통 하네스 본체 설계
2단계: Claude Code adapter 생성
3단계: Codex adapter 생성
4단계: 상호 검토
5단계: 같은 입력으로 A/B 테스트
6단계: drift 방지 규칙 고정
```

핵심은:

| Claude Code와 Codex에게 같은 일을 두 번 시키지 말고, 각자 잘 아는 부분만 맡긴다.

3. 실제 구축 순서

1단계: 공통 하네스 본체부터 만든다

먼저 이것만 만듭니다.

```
investment-research-harness/  
├── harness/  
│   ├── README.md  
│   ├── ORCHESTRATOR.md  
│   ├── contracts/  
│   ├── procedures/  
│   ├── rubrics/  
│   └── schemas/  
└── artifacts/  
    └── runs/
```

여기에는 Claude/Codex 전용 내용이 들어가면 안 됩니다.

폴더	들어갈 내용
<code>harness/ORCHESTRATOR.md</code>	Phase 0-7 전체 흐름
<code>harness/contracts/</code>	각 단계의 목적, 입력, 출력, 완료 기준
<code>harness/procedures/</code>	분석 절차, 금지사항, 방법론
<code>harness/rubrics/</code>	평가 기준
<code>harness/schemas/</code>	산출물 형식
<code>artifacts/runs/</code>	실행 결과 저장

이 단계는 Claude Code로 만들어도 되고, Codex로 만들어도 됩니다.

중요한 것은 프롬프트에서 “Claude/Codex 전용 adapter는 아직 만들지 말라”고 제한하는 것입니다.

2단계: Claude Code에게 Claude adapter만 만들게 한다

그다음 Claude Code에게는 이것만 맡깁니다.

```
CLAUDE.md  
.claude/  
  settings.json  
  settings.local.json  
agents/  
  source-collector.md  
  financial-fact-agent.md  
  valuation-agent.md  
  moat-agent.md  
  red-team-agent.md  
  final-review-agent.md
```

이때 Claude Code에게 주는 역할은:

```
너는 전체 하네스를 새로 설계하지 말고,  
이미 있는 harness/contracts, procedures, rubrics, schemas를 읽고  
Claude Code에서 호출 가능한 sub-agent adapter만 만들어라.  
공통 업무 규칙을 .claude/agents 안에 길게 복사하지 말고,  
반드시 공통 contract 경로를 참조하라.
```

Claude Code sub-agent는 `description`, `tools`, `model` 같은 Claude 전용 메타데이터가 중요합니다. Claude Code 문서상 subagent는 Markdown 파일과 YAML frontmatter로 정의되고, `description`, `tools`, `model` 같은 설정을 가질 수 있습니다. (Claude Code)

3단계: Codex에게 Codex adapter만 만들게 한다

그다음 Codex에게는 이것만 맡깁니다.

```
AGENTS.md
.codex/
  config.toml
.agents/
  skills/
    source-collection/
      SKILL.md
    financial-fact-layer/
      SKILL.md
    valuation-scenario/
      SKILL.md
    red-team-review/
      SKILL.md
```

Codex에게도 같은 원칙을 줍니다.

```
전체 하네스를 새로 설계하지 말고,
이미 있는 harness/contracts, procedures, rubrics, schemas를 읽고
Codex에서 실행 가능한 AGENTS.md와 Skills adapter만 만들어라.
공통 업무 규칙을 SKILL.md에 길게 복붙하지 말고,
반드시 공통 contract/procedure 경로를 참조하라.
```

Codex Skills는 reusable workflow를 만들기 위한 형식이며, `SKILL.md`는 `name`과 `description`을 가져야 합니다. Codex는 skill description을 보고 명시적 또는 암묵적으로 skill을 활성화할 수 있습니다. ([OpenAI Developers](#))

4. 그럼 누가 처음 뼈대를 만들어야 하나?

내 추천

현재 네 상황에서는 **Claude Code**로 먼저 공통 본체를 만들고, 이후 Codex로 Codex adapter를 검토·보완하는 방식이 가장 현실적입니다.

이유는:

이유	설명
이미 Cursor + Claude Code 환경에서 하네스를 만들고 있음	현재 작업 흐름을 유지하는 게 시행착오가 적음
Claude Code는 파일 생성·수정·프로젝트 탐색에 익숙함	skeleton 만들기 편함
처음 만들 대상은 “공통 본체”이지 Codex adapter가 아님	Claude가 만들더라도 중립 구조로 제한 가능
Codex는 이후 자기 adapter 검토에 투입	Codex 방식 오류를 줄일 수 있음

다만 조건이 있습니다.

Claude Code에게 처음부터 `.claude`, `.codex`, `.agents` 까지 한 번에 만들게 하지 마세요.

처음에는 공통 본체만 만들게 하는 게 좋습니다.

5. 실제 작업 흐름 예시

1차 작업: Claude Code로 공통 본체 생성

요청 범위:

```
harness/
artifacts/
README.md
```

금지 범위:

```
.claude/  
.codex/  
.agents/  
CLAUDE.md  
AGENTS.md
```

이 단계에서 중요한 것은 “업무 의미”를 고정하는 것입니다.

2차 작업: Claude Code로 Claude adapter 생성

요청 범위:

```
CLAUDE.md  
.claude/settings.json  
.claude/settings.local.json  
.claude/agents/*.md
```

금지 범위:

```
harness/contracts 수정 금지  
harness/procedures 수정 금지  
Codex 파일 생성 금지
```

3차 작업: Codex로 Codex adapter 생성

요청 범위:

```
AGENTS.md  
.codex/config.toml  
.agents/skills/*/SKILL.md
```

금지 범위:

```
harness/contracts 수정 금지  
harness/procedures 수정 금지  
.claude/agents 수정 금지
```

4차 작업: Codex로 공통 본체 검토

Codex에게 이렇게 시킵니다.

```
수정하지 말고 검토만 해라.  
이 하네스의 공통 contract/procedure가 Codex에서 실행하기 어려운 부분을 찾아라.  
Codex adapter와 공통 contract 사이의 불일치를 표로 보고하라.
```

5차 작업: Claude Code로 Claude adapter 검토

Claude Code에게도 이렇게 시킵니다.

```
수정하지 말고 검토만 해라.  
.claude/agents가 Claude Code sub-agent 규칙에 맞는지 확인하라.  
각 sub-agent가 공통 contract를 중복하지 않고 참조하는지 확인하라.
```

6. 최종적으로는 이런 구조가 됩니다

```

investment-research-harness/
├── README.md
├── CLAUDE.md
├── AGENTS.md
├── harness/
│   ├── ORCHESTRATOR.md
│   ├── contracts/
│   ├── procedures/
│   ├── rubrics/
│   └── schemas/
├── .claude/
│   ├── settings.json
│   ├── settings.local.json
│   └── agents/
├── .codex/
│   └── config.toml
├── .agents/
│   └── skills/
└── artifacts/
    └── runs/

```

중요한 규칙은 이겁니다.

위치	수정 책임
<code>harness/</code>	공통 업무 의미. 신중하게 수정
<code>.claude/</code>	Claude Code adapter만 수정
<code>.codex/</code>	Codex 실행 설정만 수정
<code>.agents/skills/</code>	Codex Skills adapter만 수정
<code>artifacts/</code>	실행 결과만 저장

7. “한 하네스 안에서 두 모델이 수정하면 위험하지 않나?”

맞습니다. 위험합니다.

그래서 수정 권한을 분리해야 합니다.

좋은 운영 규칙

작업	허용 도구	수정 가능 영역
공통 contract 설계	Claude Code 또는 사용자 직접	<code>harness/</code>
Claude sub-agent 생성	Claude Code	<code>.claude/</code> , <code>CLAUDE.md</code>
Codex skill 생성	Codex	<code>.codex/</code> , <code>.agents/</code> , <code>AGENTS.md</code>
검토	둘 다 가능	기본은 수정 금지
실행 결과 저장	둘 다 가능	<code>artifacts/runs/</code>
공통 규칙 변경	사람 승인 후	<code>harness/</code>

이렇게 하면 같은 하네스 안에서도 충돌을 줄일 수 있습니다.

8. 가장 중요한 장치: “수정 금지 경계선”을 문서에 박아야 함

`README.md`, `CLAUDE.md`, `AGENTS.md` 에 이런 규칙을 넣는 게 좋습니다.

```

Repository modification boundaries:

- Common workflow truth lives under /harness.
- Claude Code adapters live under /.claude and CLAUDE.md.
- Codex adapters live under /.codex, /.agents, and AGENTS.md.
- Do not duplicate business rules from /harness into adapter files.
- Adapter files must reference /harness contracts and procedures.

```

- Do not modify another platform's adapter unless explicitly requested.
- Review tasks must report findings first and must not edit files unless asked.

이 규칙이 없으면 Claude나 Codex가 “좋은 의도”로 다른 도구용 파일까지 손댈 수 있습니다.

9. drift를 막는 방법

9-1. manifest 파일을 둔다

harness/MANIFEST.md

내용은 이런 식입니다.

Phase	Contract	Procedure	Claude Agent	Codex Skill	Output
00 Intake	00_intake.contract.md	intake.md	intake-agent.md	intake/SKILL.md	00_intake_output.md
01 Source Collection	01_source_collection.contract.md	source_collection.md	source-collector.md	source-collection/SKILL.md	01_source_collection_output.md
02 Fact Layer	02_fact_layer.contract.md	financial_fact_layer.md	financial-fact-agent.md	financial-fact-layer/SKILL.md	02_fact_layer_output.md

이걸 보면 어느 파일이 어느 역할을 하는지 한눈에 알 수 있습니다.

9-2. adapter 파일에 버전 태그를 둔다

예:

```
contract_version: 2026-05-31
contract_ref: harness/contracts/02_fact_layer.contract.md
procedure_ref: harness/procedures/financial_fact_layer.md
```

이러면 나중에 공통 contract를 바꿨을 때 adapter가 구버전인지 확인하기 쉽습니다.

9-3. adapter에는 “업무 규칙”을 길게 쓰지 않는다

나쁜 예:

.claude/agents/financial-fact-agent.md 안에
재무제표 수집 기준, 음수 표기 규칙, 출처 규칙, valuation table 구조를 전부 다시 작성

좋은 예:

.claude/agents/financial-fact-agent.md 안에는
“harness/contracts/02_fact_layer.contract.md를 읽고 따르라”
“harness/procedures/financial_fact_layer.md를 읽고 따르라”
정도만 작성

10. 네가 실제로 Claude Code에 줄 첫 요청은 이렇게 가는 게 좋음

첫 요청은 전체를 다 만들라는 요청이 아니라, 공통 본체만 만들라는 요청이어야 합니다.

내가 만들고 싶은 것은 Claude Code와 Codex가 모두 사용할 수 있는 모델 중립형 investment-research-harness다.

이번 작업에서는 Claude Code 전용 파일과 Codex 전용 파일을 만들지 말고, 공통 하네스 본체만 만들어라.

생성 범위:

- README.md
- harness/ORCHESTRATOR.md
- harness/MANIFEST.md

- harness/contracts/
- harness/procedures/
- harness/rubrics/
- harness/schemas/
- artifacts/runs/.gitkeep

금지:

- CLAUDE.md 생성 금지
- AGENTS.md 생성 금지
- .claude/ 생성 금지
- .codex/ 생성 금지
- .agents/ 생성 금지

원칙:

- /harness는 single source of truth다.
- 각 contract는 목적, 입력, 출력, 금지사항, 완료 기준을 포함해야 한다.
- procedures는 업무 방법론만 담고 특정 모델이나 도구 이름을 포함하지 않는다.
- 모든 산출물은 artifacts/runs/{run_id}/ 아래에 저장되도록 설계한다.

이렇게 하면 Claude Code가 자기 방식으로 과도하게 끌고 가는 걸 막을 수 있습니다.

11. 그다음 Claude Code adapter 요청

이제 기존 /harness 공통 본체를 기준으로 Claude Code adapter만 만들어라.

생성 범위:

- CLAUDE.md
- .claude/settings.json
- .claude/settings.local.json
- .claude/agents/*.md

금지:

- /harness 수정 금지
- AGENTS.md 생성 금지
- .codex/ 생성 금지
- .agents/ 생성 금지

규칙:

- .claude/agents 파일에는 업무 규칙을 길게 복붙하지 말고, 반드시 /harness/contracts와 /harness/procedures를 참조하라.
- 각 sub-agent는 YAML frontmatter를 포함해야 한다.
- name, description, tools, model을 명시하라.
- description에는 언제 이 sub-agent를 사용해야 하는지 분명히 적어라.
- 각 sub-agent는 자기 phase의 output file만 작성하도록 제한하라.

12. 그다음 Codex adapter 요청

Codex에는 이렇게 요청하는 게 좋습니다.

이 repository는 이미 /harness에 모델 중립형 공통 하네스 본체가 있다.

이번 작업에서는 Codex adapter만 만들어라.

생성 범위:

- AGENTS.md
- .codex/config.toml
- .agents/skills/*/SKILL.md

금지:

- /harness 수정 금지
- CLAUDE.md 수정 금지
- .claude/ 수정 금지

규칙:

- Codex Skills는 /harness/contracts와 /harness/procedures를 참조해야 한다.
- 공통 업무 규칙을 SKILL.md에 길게 복붙하지 않는다.
- 각 SKILL.md는 name과 description을 포함해야 한다.
- description은 언제 해당 skill이 활성화되어야 하는지 명확히 적는다.
- Codex가 명시적으로 skill을 호출하거나, task가 description과 맞을 때 사용할 수 있도록 작성한다.

Codex의 subagent는 Claude Code와 달리 명시 요청 시 생성되는 구조에 가깝고, Codex 문서는 subagent가 병렬 특화 agent를 spawn하고 결과를 모으는 방식이며, 현재는 명시적으로 요청해야 새 agent를 spawn한다고 설명합니다. 또한 subagent마다 모

델·도구 작업을 하므로 단일 agent보다 토큰을 더 쓴다고 안내합니다. (OpenAI Developers)

13. 내가 추천하는 최종 운영 방식

구축 단계

단계	실행 도구	목적	수정 영역
1	Claude Code 또는 사용자	공통 본체 생성	<code>harness/</code> , <code>artifacts/</code>
2	Claude Code	Claude adapter 생성	<code>CLAUDE.md</code> , <code>.claude/</code>
3	Codex	Codex adapter 생성	<code>AGENTS.md</code> , <code>.codex/</code> , <code>.agents/</code>
4	Codex	Codex 관점 검토	원칙상 수정 금지
5	Claude Code	Claude 관점 검토	원칙상 수정 금지
6	사용자	차이점 승인	필요한 파일만 수정
7	둘 다	같은 입력으로 실행 테스트	<code>artifacts/runs/</code>

평소 수정 단계

바꾸고 싶은 것	수정 위치
분석 절차 변경	<code>harness/procedures/</code>
산출물 형식 변경	<code>harness/schemas/</code>
평가 기준 변경	<code>harness/rubrics/</code>
단계 흐름 변경	<code>harness/ORCHESTRATOR.md</code> , <code>harness/MANIFEST.md</code>
Claude sub-agent 호출 조건 변경	<code>.claude/agents/</code>
Codex skill trigger 변경	<code>.agents/skills/*/SKILL.md</code>
모델/권한 변경	<code>.claude/settings</code> , <code>.codex/config.toml</code>

최종 결론

네가 같은 작업을 두 번 할 필요는 없습니다.

하지만 한 모델에게 전체를 다 맡기는 것도 좋지 않습니다.

가장 좋은 방식은:

공통 하네스 본체는 한 번만 만들고, Claude Code와 Codex는 각자 자기 실행 adapter만 만들거나 검토하게 하는 것입니다.

추천 순서는:

1. Claude Code로 공통 `/harness` 본체만 생성
2. Claude Code로 `.claude/agents` adapter 생성
3. Codex로 `AGENTS.md`, `.codex`, `.agents/skills` adapter 생성
4. 각 도구가 자기 adapter만 검토
5. 공통 contract는 사람이 승인한 뒤에만 수정

이렇게 하면 “하나의 하네스”를 유지하면서도, Claude Code와 Codex의 서로 다른 실행 방식을 안전하게 수용할 수 있습니다.